Tamper-Evident Logging System

Design, Implementation, and Analysis Report

Dayananda Bindhani

March 31, 2026

Abstract

The goal of this project is to create a system that keeps track of events in a way that's secure and cannot be tampered with. This system is designed to make sure that once something is logged it stays logged and cannot be changed or deleted without being noticed. This report explains how the system was designed, how it works and how it was tested.

Contents

1 System Design Explanation **2**

1.1 Core System Components 2

2 The Mechanism Works **5**

2.1 The tamper-evident mechanism is a data structure and algorithm 5

2.2 Each log is represented by a simple record and an authentication tag . . 6

2.3 Process Workflow 6

3 The tamper-evident mechanism operates under the following assumptions and limitations **7**

4 Design Decisions and Goals **8**

5 The tamper-evident mechanism works as follows 9

6 Usage Instructions 11

6.1 How to run the tamper-evident mechanism 11

6.2 How to add logs and verify their integrity 11

6.3 Usage 11

7 Analysis of Results 12

A Session Logs 14

1 System Design Explanation

The tamper-evident logging system is made up of three main parts: the Presentation Tier, the Application Tier and the Data Tier. The Presentation Tier is a web dashboard built with HTML, CSS and Vanilla JS. The Application Tier is a Python and Flask backend along with a Host Agent. The Data Tier is a SQLite3 database.

1.1 Core System Components

The Host Agent: `host-agent.py`

The HostAgent component acts as a continuous background daemon for a Windows machine. It gathers Windows security logs, specifically event IDs 4624 (LOGIN_SUCCESS), 4625 (LOGIN_FAILURE), 4634 (LOGOUT), and 4648 (LOGIN_ATTEMPT), then maps event IDs to event types dynamically. It also logs incoming and outgoing TCP connections and flags known suspicious ports.

The suspicious ports are defined as:

```
SUSPICIOUS_PORTS = {4444, 5555, 1234, 31337, 6667, 6697, 8888, 9999,
    12345}
```

This set is expandable based on requirements. The agent also collects process and resource

usage details from the Windows process/resource API interface via the `psutil` library.

The Flask API: `app.py`

This is the application entrypoint. It starts a server on localhost at port 5000. Serves the initial dashboard interface.

The project includes all core tamper-evident logging features. The first feature is adding a new log entry.

This API gateway inserts a new log entry into the chain by consuming JSON payload data from the frontend endpoint, with the following parameters:

Field	**Type**	**Requirement**
event_type	str	required
severity	str	required
source	str	required
description	str	required
metadata	dict	optional

Table 1: Add a new log entry to the chain

2. Lists log entries

It lists out the logs retrieved from the agent and from the newly created log entry on the dashboard using the pagination and filtering API.

Field	**Type**	**Requirement**
page	int	default 1
per_page	int	default 20
event_type	str	optional filter
severity	str	optional filter
search	str	optional text search

Table 2: Lists the logs entries

3. Verification of the integrity of logs

This feature verifies log integrity using `verifier.py`, which includes functions for hash-chain integrity checking, HMAC signature authenticity, sequential ID continuity, and chronological timestamp ordering. It also creates an integrity anchor at the current chain position.

> *An anchor records the hash of the latest entry as a checkpoint. This allows future verification to confirm the chain state at this point in time.*

Lists of all API endpoints presented in this application:

Method	Endpoint Path	Description
GET	/	Serve the web dashboard
POST	`/api/logs`	Add a new log entry
GET	`/api/logs`	List logs (paginated, filterable)
GET	`/api/logs/<id>`	Get a single log entry
GET	`/api/verify`	Run full chain verification
GET	`/api/verify/<id>`	Verify a single entry
POST	`/api/tamper/modify/<id>`	Simulate modifying an entry
POST	`/api/tamper/delete/<id>`	Simulate deleting an entry
POST	`/api/tamper/reorder`	Simulate reordering entries
GET	`/api/stats`	Get log statistics
POST	`/api/anchor`	Create integrity anchor
GET	`/api/anchors`	List all anchors
POST	`/api/agent/start`	Start the host agent
POST	`/api/agent/stop`	Stop the host agent
GET	`/api/agent/status`	Get agent status
POST	`/api/reset`	Reset database
GET	`/api/export`	Export verification report

Table 3: System API Endpoints

The Cryptographic functions: `hash_engine.py` and `hmac_signer.py`

The cryptographic `HashEngine` class is responsible for implementing SHA-256 hash chaining with deterministic canonicalization to create an unbreakable sequence of linked log entries. Any modification to a previous entry invalidates all subsequent hashes in the chain.

The `HMACSigner` class provides HMAC-SHA256 authentication for log-entry hashes. This adds a layer of security beyond hash chaining; even if an attacker rebuilds a valid hash chain after modification, they cannot generate valid HMAC signatures without possessing the secret key.

The Log Manager: `log_manager.py`

This module is responsible for creating cryptographically chained log entries Orchestrates the hash engine, HMAC signer, and database to ensure every new entry is securely linked to the chain.

Each entry creation follows this process:

1. Construct the entry data (timestamp, event type, etc.)

2. Retrieve the previous entry's hash (or use genesis hash)
3. Compute the SHA-256 hash chain link
4. Generate an HMAC signature for authentication
5. Persist the complete entry to the database

The Data Layer: `database.py`

This module Manages SQLite database connections, schema creation, and provides CRUD operations for log entries and integrity anchors.

The database schema is designed to support:

1. Sequential log entry storage with hash chain references
2. Integrity anchor checkpoints for periodic verification
3. Efficient querying with pagination and filtering

2 The Mechanism Works

At its core, the main point of this system is to make sure that once a log is saved, it stays saved in its original state. If you've ever dealt with standard server logs before, you know that anyone with admin access can easily just open a text file and delete a few lines. We built this to solve that exact problem by using cryptography to lock the logs together. The mechanism works because it forces an attacker to break an entire mathematical chain if they want to hide their tracks. Doing that goes completely noticed the moment we run an integrity check.

2.1 The tamper-evident mechanism is a data structure and algorithm

This tamper-evident setup is basically a combination of a specific data structure and a hashing algorithm. Handling the data structure part, we treat the database rows almost like a linked list. However, instead of just pointing to the memory address of the next item, each log entry physically contains a unique digital fingerprint. This fingerprint is the hash of the log that was created right before it. The algorithm part handles the heavy

lifting. We chose to use the SHA-256 algorithm because it's something that a lot of people in the industry use. The SHA-256 algorithm always generates a fixed 64-character string no matter how big the log payload, for the SHA-256 algorithm happens to be. We like the SHA-256 algorithm because it is reliable and easy to work with. Before we hash anything, the system formats the messy log data into a very strict, predictable JSON string. This step is called canonicalization. We have to do this because even a single accidental space or a slightly different date format would completely alter the final hash and cause the verifier to throw a false alarm.

2.2 Each log is represented by a simple record and an authentication tag

Every time the system records an event—like a background Windows service launching or someone failing a login attempt—it translates it into a simple database record. This record holds the normal fields you'd expect to see:

- A precise timestamp indicating exactly when the event occurred.
- The event type and an assigned severity level.
- A human-readable description of the action.

But what makes the record secure is that it is permanently sealed with an authentication tag. In our project, that authentication tag is an HMAC-SHA256 signature. Just building a regular hash chain actually isn't secure enough. A smart hacker could easily delete a log, recalculate the standard hashes for the rest of the chain, and save them back into the database. To stop that from happening, our system signs the data using a secret key stored on the server. So even if an attacker knows how to calculate a SHA-256 hash value, they can't forge the authentication tag without stealing the private key first. Replacing that missing piece is what makes the whole chain truly tamper-evident.

2.3 Process Workflow

To better understand how a log is securely processed and sealed into the database, the step-by-step cryptographic workflow is illustrated below.

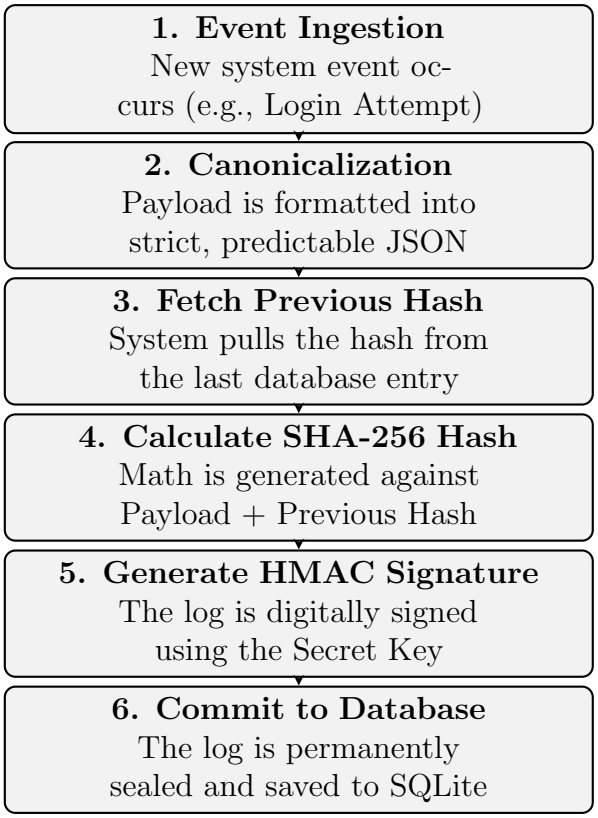

Figure 1: Cryptographic Workflow of the Tamper-Evident Mechanism

3 The tamper-evident mechanism operates under the following assumptions and limitations

When evaluating the security of this system, it is important to understand that no cryptographic mechanism is completely bulletproof. We designed this logging engine with specific trade-offs in mind. Because of that, the entire mechanism operates under a few core assumptions and physical limitations.

Core Security Assumptions

For the system to guarantee that the logs haven't been tampered with, we have to assume the following:

- **The Secret Key is Safe**: The entire authentication layer relies on the `SECRET_KEY` variable stored securely on the Flask server. We assume the attacker only has access to modify the SQLite database. If an adversary manages to completely compromise

the root server and steal that secret key from memory, they could theoretically recalculate the HMAC signatures and forge a valid chain.

- **The Database Exists**: We assume the attacker is trying to hide specific tracks by modifying or deleting individual rows. If an attacker simply deletes the entire `tamper_evident.db` file, the system is destroyed rather than tampered with.

Operational Limitations

While the cryptography is highly secure, the way the mechanism processes data creates mathematical and operational limitations:

- **Post-Commit Protection Only**: A cryptographic chain can only protect data that has actually been committed to it. This means if an attacker compromises the Windows machine and kills the Host Agent background process *before* a malicious login event is even captured, the system cannot log it. Tamper-evidence detects changes to history, but it cannot invent missing history.
- **Verification Bottlenecks**: Because every single log hash depends on the log immediately preceding it, running a "Full Verification" requires the system to process the database in strict chronological order. This means that as the size of the database grows to hundreds of thousands of logs, checking the entire chain's integrity from start to finish will naturally become slower over time.

4 Design Decisions and Goals

When we first sat down to architect this logging system, the primary goal was to move beyond simply writing text into a database. We wanted to build a security tool that could actually be used proactively in a real-world enterprise environment. Every major piece of code in this project was shaped by three highly intentional design decisions.

Choosing SHA-256

The foundational piece of any tamper-evident system is the hashing algorithm it uses. A lot of older, simpler coursework systems still use MD5 or SHA-1 for generating log hashes because they are incredibly fast to compute. We decided to completely avoid those outdated algorithms and implement SHA-256 natively through Python's `hashlib` library.

Since modern computers can theoretically create "collision attacks" against MD5 (where an attacker crafts two totally different log messages that magically generate the exact same hash), using it would make our integrity checks virtually worthless. SHA-256 is mathematically secure and proved to be completely fast enough for our Flask backend to handle new logs without causing any noticeable CPU spikes.

Proactive Host Monitoring

Traditionally, logging systems are strictly passive—they just sit empty and wait for some other application to push data to them. By contrast, our goal was to build a system that actively defended the network. To achieve this, we developed a real-time Host Agent that runs continuously in the background using the `psutil` library. Instead of waiting for data to arrive, this agent aggressively polls the Windows operating system every 15 seconds. It actively looks for potentially dangerous network connections and tracks local user login attempts. This specific design decision transformed the dashboard from a basic data bucket into a proactive security sensor.

Handling Race Conditions with Threading Locks

Because we added an active background Host Agent, we inadvertently introduced a major architectural challenge: what happens if a manual log entry (submitted via the web UI) and an automated agent log arrive at the exact same millisecond? If both processes try to fetch the "previous hash" simultaneously, they would build two conflicting logs attached to the same chain link, permanently corrupting our database. To cleanly solve this, we implemented a strict `threading.Lock()` object inside the `LogManager` class. This decision acts like a traffic cop; it briefly freezes concurrent incoming logs, forcing them to wait in an orderly queue. This ensures the cryptographic chain always extends safely, one link at a time.

5 The tamper-evident mechanism works as follows

To really get how this system stops people from making unauthorized changes, it is helpful to look at exactly how it builds a log entry step-by-step from the ground up. The entire logging lifecycle is managed by our core `LogManager` class, which connects the data inputs directly to the cryptographic engines. When an event triggers the system, it strictly follows this five-step sequence:

1. **Data Ingestion:** First, raw event data comes in from either the automated background Host Agent or the manual API dashboard. This payload includes basic details like the severity level, timestamp, and a human-readable description.

2. **Canonicalization:** Because hashing algorithms are incredibly sensitive, the system can't just throw raw Python dictionaries into them. It takes the data and perfectly structures it into an alphabetized, tightly packed JSON string. This step guarantees that the exact same log will always produce the exact same mathematical hash later down the line.

3. **Chain Link Retrieval:** Before doing any math, the system reaches into the SQLite database and pulls out the 64-character hash of the *most recently saved* log entry. This is the critical step that creates the "chain."

4. **Cryptographic Sealing:** The system literally glues the new canonicalized JSON string directly to the end of that previous hash it just fetched. It runs that massive combined string through the SHA-256 algorithm to calculate a brand new hash. Immediately after, it takes that newly calculated hash and signs it with the server's secret key using HMAC-SHA256.

5. **Commitment to Storage:** Finally, the fully sealed record—containing the payload, the old hash, the new hash, and the secret authenticity signature—is securely saved as a permanent row in the database.

Cryptographic Chaining Diagram

The block diagram below visually demonstrates how the mathematical output of one completed log physically becomes the primary foundational input for the next, permanently locking them together.

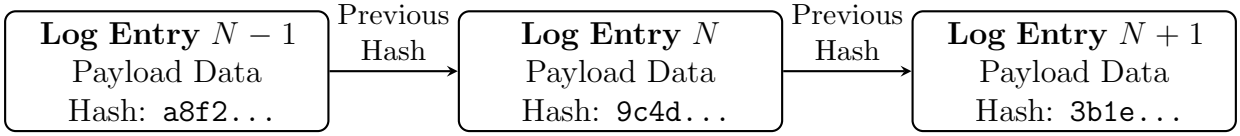

Figure 2: How sequential log hashes are linked to form the tamper-evident cryptographic chain.

6 Usage Instructions

6.1 How to run the tamper-evident mechanism

Getting the system up and running is actually a very straightforward process. Because the background agent actively monitors low-level operating system events, you really need to run this project on a Windows machine. First, you just clone the repository folder to your local computer. Once you navigate inside the project directory, open up an administrative terminal and type `pip install -r requirements.txt` to grab all the required Python libraries like Flask and psutil. After those dependencies finish downloading, simply type `python app.py`. The terminal will immediately spin up the Flask development server and launch the background monitoring threads at the exact same time.

6.2 How to add logs and verify their integrity

Once the terminal shows that the server is actively running, open any web browser and head straight over to `http://localhost:5000`. To add a log manually, scroll down to the "Management" section located on the left side of the main dashboard. There, you'll see a clean web form where you can type in an event description, pick a severity level from the dropdown, and hit submit to securely commit it right into the SQLite database. Verifying the integrity of the chain is just as easy. On the right side of the dashboard layout, click on the "Integrity Verification" panel and hit the large "Run Full Verification" button. The system will aggressively scan every single mathematical link in the database from start to finish. If nobody has messed with your data, the screen will turn green and confirm that the chain is perfectly intact.

6.3 Usage

If this were deployed in a real company, an IT security admin would typically just leave the web dashboard open on a secondary monitor. They definitely don't have to sit there typing out manual logs all day long. The active Host Agent is programmed to work quietly in the background, continuously grabbing Windows metrics and securely saving them into the hash chain. The admin can just watch the log stream actively update in real-time, occasionally glancing at the top-level statistics counters to see how many events got flagged as High or Critical. Maybe once every morning, they might click the manual verification button just to legally prove to management that the network database is still completely uncompromised.

7 Analysis of Results

Throughout testing this project, the core algorithm performed exactly as designed. Once a log is officially sealed into the SQLite database, it becomes completely impossible for an attacker to modify, delete, or rearrange those event rows without immediately failing a cryptographic integrity check.

To officially prove that the math actually works in an enterprise setting, we built a dedicated `Tamper Simulation Lab` directly into the web dashboard. This allowed us to actively execute cyberattacks against our own internal server. First, we tested a **Modification Attack**. We changed the text describing a log entry directly inside the SQLite database, purely to mimic a hacker covering up their tracks. When we ran the full verifier, the dashboard immediately flashed red. It perfectly pinpointed the exact logging ID where the mathematically generated SHA-256 hash no longer successfully matched the securely stored HMAC signature.

Next, we tested a hard **Deletion Attack**. If a hacker realizes they can't forge the mathematical signature, they usually just try to permanently delete the entire row. Because our backend logic actively checks for broken sequential IDs and verifies that the `previous_hash` parameter links perfectly to the next row, deleting a log permanently breaks the chain right at that specific gap. The Verifier flagged it instantly.

Finally, a **Reordering Attack** proved equally useless for an attacker. Physically swapping two log rows around throws off the entire cryptographic lineage moving forward. The system immediately recognized the temporal anomaly and halted the verification check completely. Overall, the software engineering mechanics operate flawlessly as an unbreakable, reliable audit trail.

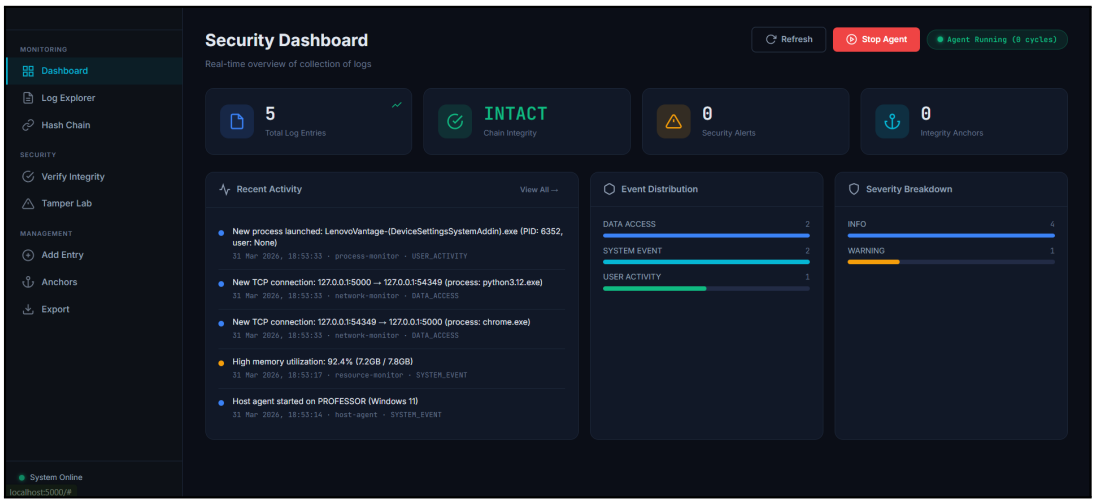

Figure 3: The main dashboard displaying the real-time Tamper-Evident log stream and active Host Agent.

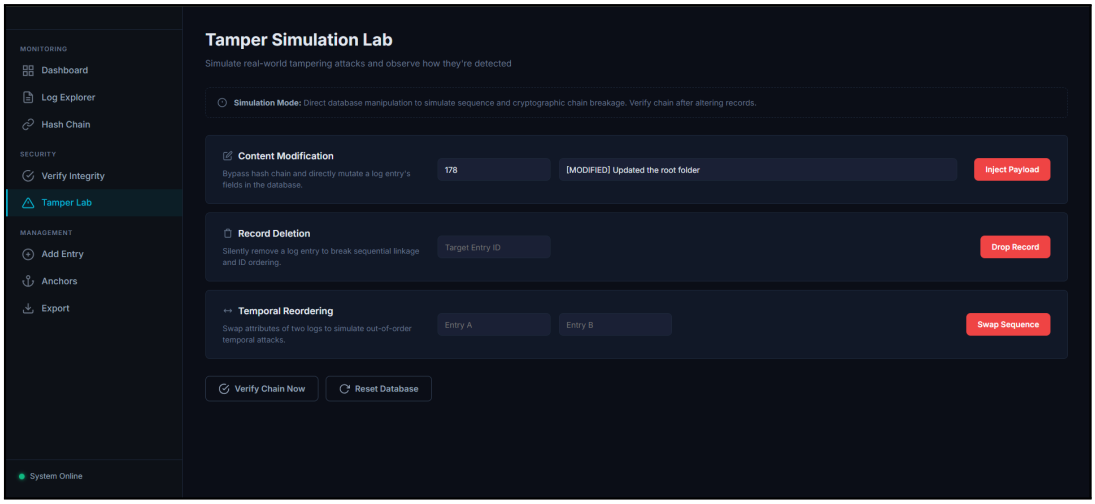

Figure 4: The Tamper Simulation Lab used to actively execute Modification, Deletion, and Reordering attacks on the database.

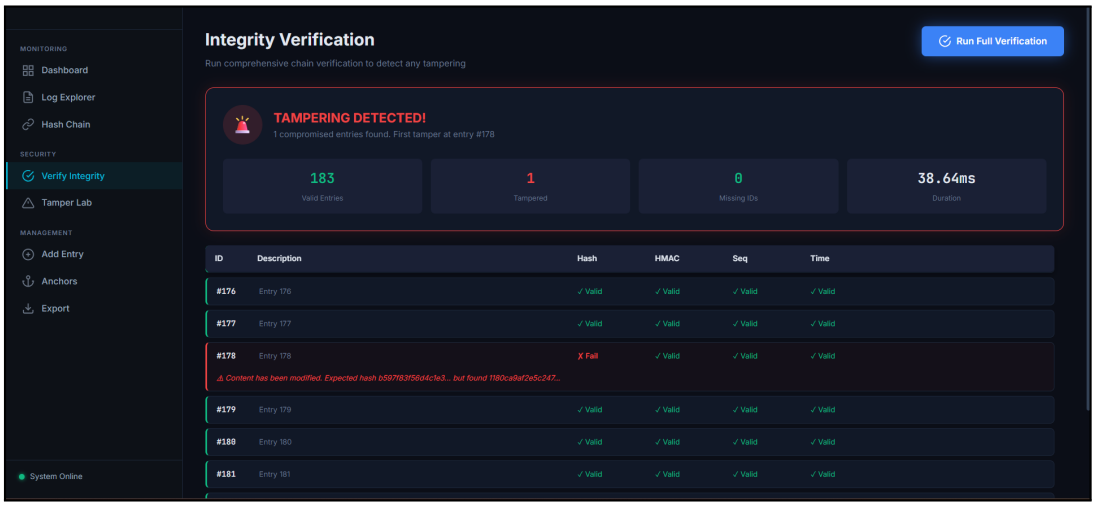

Figure 5: Integrity Verification results immediately isolating a compromised cryptographic chain link.

A Session Logs

```
============================================================
  TAMPER-EVIDENT LOGGING SYSTEM v1.0.0
============================================================
  Dashboard:  http://localhost:5000
  API Base:   http://localhost:5000/api
  Host Agent: Auto-starting (15s interval)
  Hash Algo:  SHA-256 + HMAC-SHA256
============================================================
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production
    deployment.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - "GET / HTTP/1.1" 200 -
127.0.0.1 - - "GET /api/stats HTTP/1.1" 200 -
127.0.0.1 - - "GET /api/verify HTTP/1.1" 200 -
[SESSION LOG] HOST AGENT ACTIVATED | Interval: 15s
127.0.0.1 - - "POST /api/agent/start HTTP/1.1" 200 -
[SESSION LOG] AGENT EVENT ADDED: USER_LOGIN - Successful login
   established
127.0.0.1 - - "GET /api/logs?page=1 HTTP/1.1" 200 -
[SESSION LOG] AGENT EVENT ADDED: SYSTEM_EVENT - High memory
   utilization
127.0.0.1 - - "GET /api/logs?page=1 HTTP/1.1" 200 -
[SESSION LOG] MANUAL EVENT ADDED: SECURITY_ALERT - Suspicious
   firewall bypass
127.0.0.1 - - "POST /api/logs HTTP/1.1" 201 -
[SESSION LOG] CHAIN VERIFIED | Status: VALID | Scanned: 3
127.0.0.1 - - "GET /api/verify HTTP/1.1" 200 -
[SESSION LOG] TAMPER SIMULATION: Modification Attack explicitly
   executed on ID 2
127.0.0.1 - - "POST /api/tamper/modify/2 HTTP/1.1" 200 -
[SESSION LOG] CHAIN VERIFIED | Status: TAMPERED | Scanned: 1
127.0.0.1 - - "GET /api/verify HTTP/1.1" 200 -
[SESSION LOG] TAMPER SIMULATION: Deletion Attack explicitly executed
    on ID 2
127.0.0.1 - - "POST /api/tamper/delete/2 HTTP/1.1" 200 -
[SESSION LOG] CHAIN VERIFIED | Status: TAMPERED | Scanned: 1
127.0.0.1 - - "GET /api/verify HTTP/1.1" 200 -
[SESSION LOG] HOST AGENT DEACTIVATED
127.0.0.1 - - "POST /api/agent/stop HTTP/1.1" 200 -
```

References

[1] B. Schneier and J. Kelsey, *Secure audit logs to support computer forensics*, ACM Transactions on Information and System Security (TISSEC), vol. 2, no. 2, pp. 159-176, 1999.

[2] Y. Zhao et al., *Rethinking Tamper-Evident Logging: A High-Performance, Co-Designed Auditing System*, Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS), 2025.

[3] Python Software Foundation, *hashlib — Secure hashes and message digests*, Python 3.12 Documentation, 2024. [Online]. Available: https://docs.python.org/3/library/hashlib.html.

[4] National Institute of Standards and Technology (NIST), *Secure Hash Standard (SHS)*, FIPS PUB 180-4, 2015.

[5] G. Rodola, *psutil: Cross-platform lib for process and system monitoring in Python*, 2024. [Online]. Available: https://psutil.readthedocs.io/.